

Sistemas Operacionais

Memória Virtual

Professor: **Jeremias Moreira Gomes**

E-mail: jeremias.gomes@idp.edu.br

Introdução

Memória Virtual

Memória Virtual

- É uma técnica que usa a memória secundária como uma “cache” das partes dos espaços dos processos
 - Por que Memória Virtual?
 - O tamanho do software é cada vez maior
 - Maior grau de multiprogramação
 - Permitir a execução de programas maiores que a RAM
-

Memória Virtual

- Todo processo (nos sistemas operacionais atuais) utiliza endereços virtuais e não físicos
 - A MMU é a responsável por realizar a conversão

Técnicas de Memória Virtual (1/2)

- **Paginação**

- Quebra da memória em blocos de tamanho fixo
 - Usualmente 4KB
- O espaço de endereçamento virtual é dividido em páginas virtuais

Técnicas de Memória Virtual (2/2)

- **Segmentação**
 - Blocos de tamanho arbitrário chamados de segmentos
 - Contém mesmo tipo de informações
 - Dados, pilha, etc

Técnicas de Memória Virtual

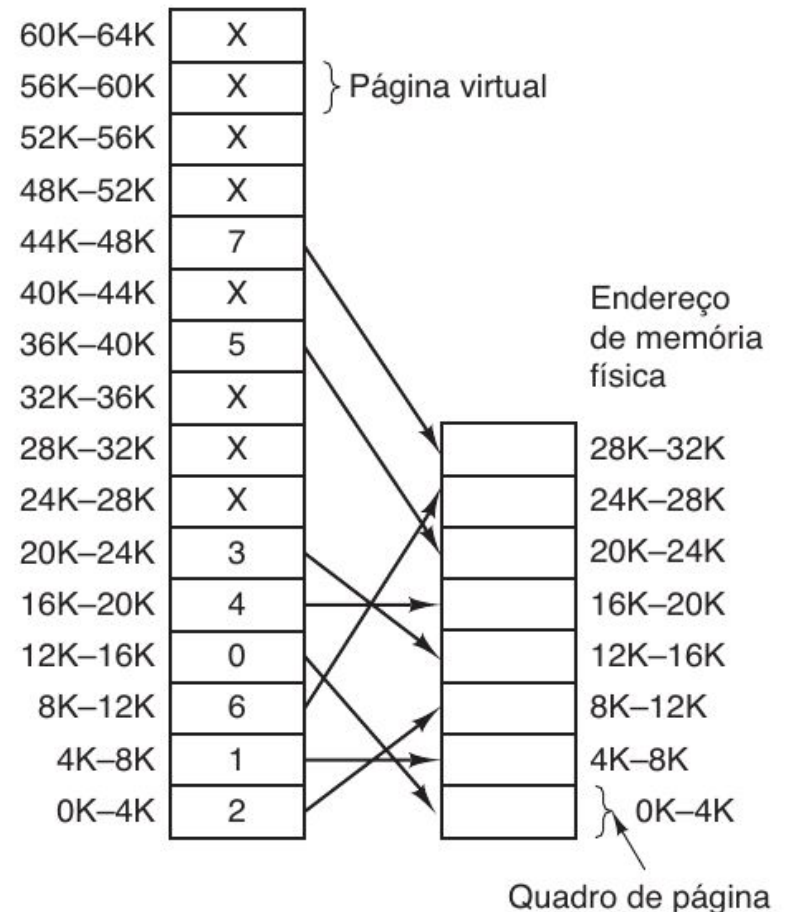
- Na prática, os sistemas utilizam uma mistura das duas técnicas:
 - Você pagina a memória, e cada página é delimitada a um mesmo tipo de informação e permissão de acesso/uso
 - Para ambos os casos, a MMU continua fazendo o mapeamento entre endereços reais e virtuais
-

Paginação

- **Páginas** - Unidades de tamanho fixo no dispositivo secundário
- **Frames** - Unidades correspondentes na memória física
- **Page fault** - É o evento quando uma página que não está na RAM é referenciada
- **Tabela de Páginas** - Estrutura para mapear página-frame

Exemplo de Paginação

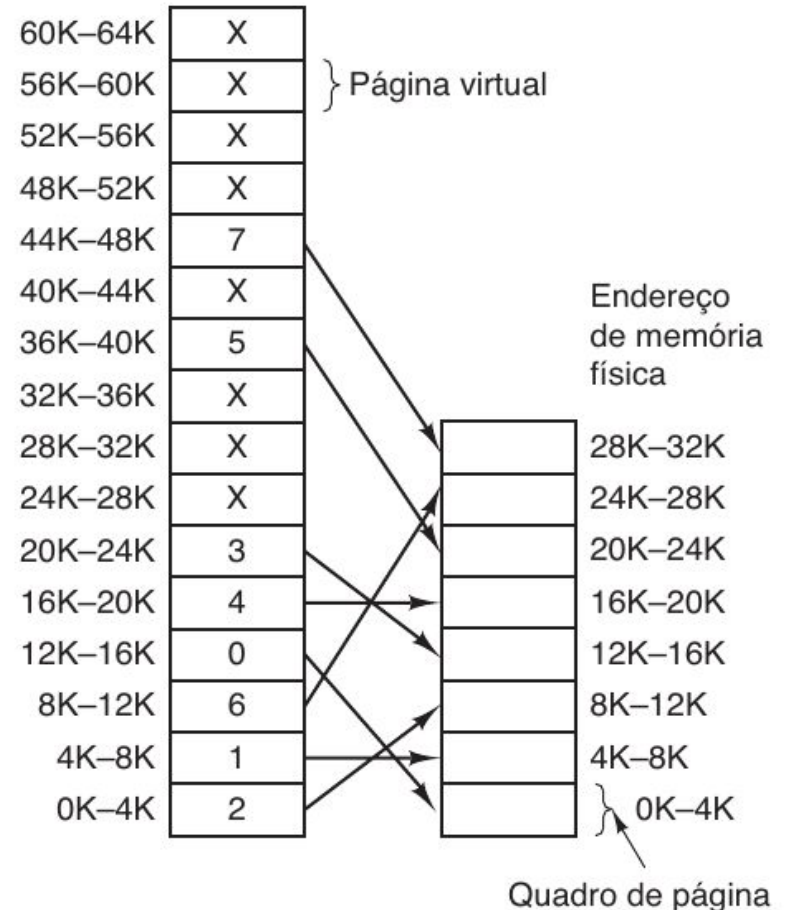
- 64K de endereços virtuais
 - Endereços de 16 bits
- 32K de memória física
- Cada página é múltiplo de 4096
 - Cada frame também (==)
 - Em linux há páginas de 4K, 2MB e 1GB
- MMU faz o mapeamento



Exemplo de Paginação

```
_start:
    mov rax, [0]
```

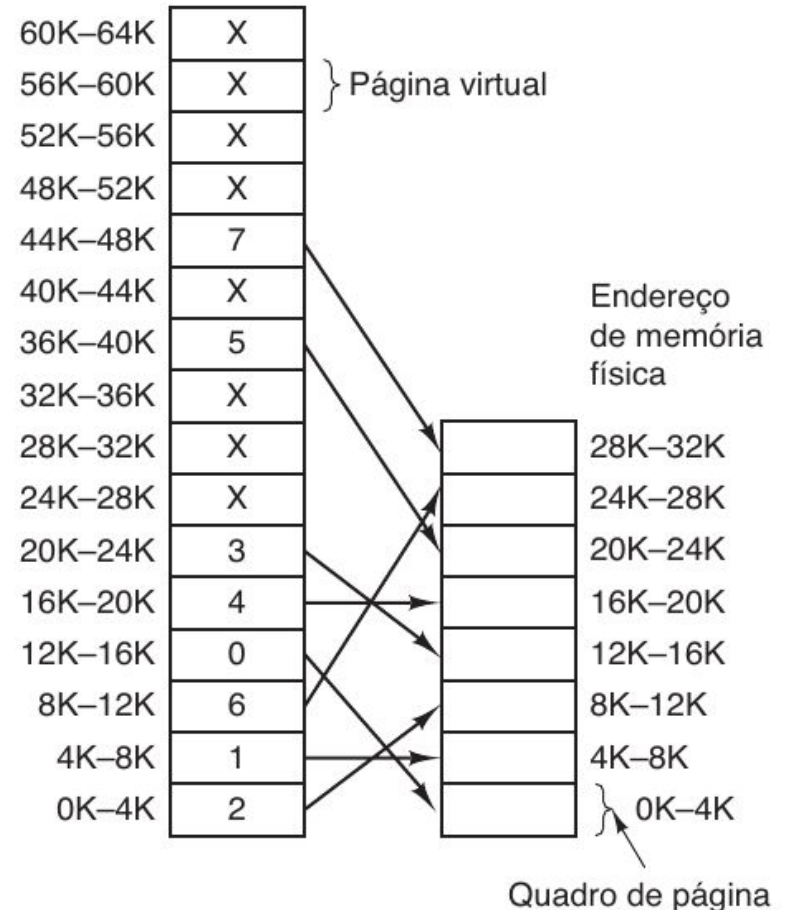
- O endereço virtual é 0
- Enviado para a MMU
 - Detecta que é a página 0 (+0 bytes)
 - Acha o frame correspondente 2
 - Soma 8192 ao endereço virtual
 - Coloca no barramento 8192



Exemplo de Paginação

```
_start:
    mov rbx, [20500]
```

- O endereço virtual é 20500
- Enviado para a MMU
 - Detecta que é a página 5 (+20 bytes)
 - Acha o frame correspondente 3
 - Subtrai 20480 (20) e soma 12288 (12308)
 - Coloca no barramento 12308



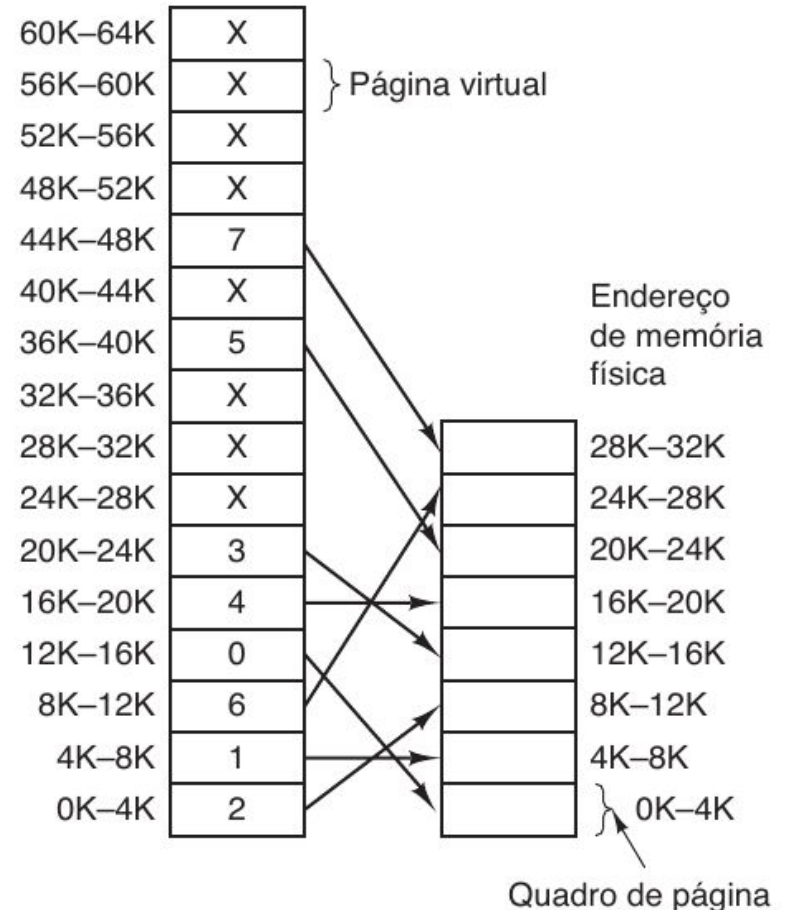
Paginação

- Do ponto de vista da memória, esta nem percebe o trabalho da MMU, apenas recebe a solicitação do endereço físico
- Nos exemplos, há 16 páginas e 8 frames
 - Não resolve o problema do espaço virtual ser maior
 - Somente 8 páginas por vez mapeadas
 - Se não está mapeada*, aparece com um X

Exemplo de Paginação

```
_start:
    mov rcx, [32780]
```

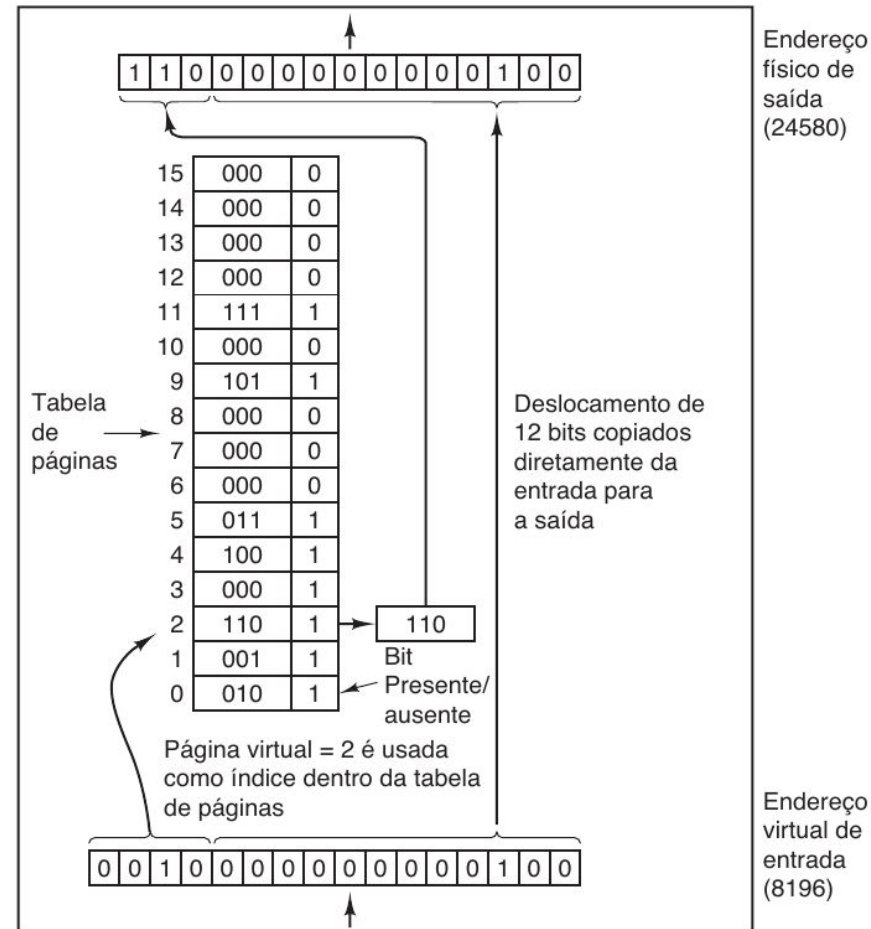
- O endereço virtual é 32780
- Enviado para a MMU
 - Detecta que é a página 8 (+12 bytes)
 - Descobre que a página não está mapeada
 - MMU desvia (interrupção) para o SO (**page fault**)
- SO escolhe um frame para carregar do disco
 - Substitui marcando a página que saiu com X
 - Relaciona a página solicitada com o frame
- MMU coloca no barramento memória + offset



Paginação - MMU (1/2)

- A MMU utiliza como tamanho de páginas potências de 2
 - Exemplo com endereço virtual 8196 (1/2):
 - `0010000000000100` (chega na MMU)
 - 4 primeiros bits (número da página) (`0010 == 2`)
 - 12 últimos bits (deslocamento) (`00000000000100 == 8`)
 - Número da página é o índice em uma **tabela de páginas** que resulta em um frame (quadro) se o bit presente/ausente for 1

Paginação - MMU



Paginação - MMU (2/2)

- A MMU utiliza como tamanho de páginas potências de 2
 - Exemplo com endereço virtual 8196 (2/2):
 - Se o bit presente/ausente for 0, é nesse momento que ocorre a interrupção para o SO (page fault)
 - Nos dois casos (presente ou ausente), após ter a memória carregada, o valor na tabela 3 bits, correspondem ao respectivo frame e o offset possui os últimos 12 bits do frame

Tabela de Páginas

- Quando um processo executa, ele manipula endereços lógicos
- Para um programa funcionar, é necessário transformar os endereços lógicos do programa no endereço físico correspondente que é feito por meio da **Tabela de Páginas**
- O propósito da Tabela de Páginas é mapear as páginas virtuais em quadros de páginas (o mesmo que frames)
 - É uma função matemática que recebe como argumento um endereço virtual e retorna o endereço físico

Tabela de Páginas - Componentes de um Endereço

- **Número de página (p)** - utilizado como índice para a tabela de páginas
- **Offset da página (d)** - utilizado, em conjunto com o endereço base, para calcular e definir o endereço de memória físico



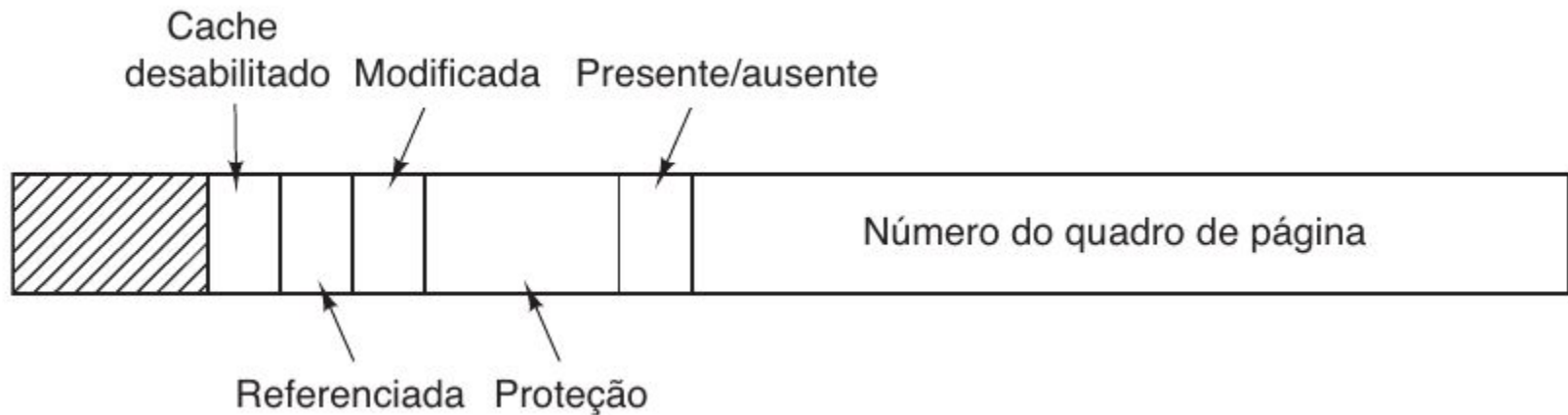
Tabela de Páginas

- **Páginas Grandes**
 - Leitura mais eficiente, tabela menor, fragmentação interna
- **Páginas Menores**
 - Leitura menos eficiente, tabela maior, menos fragmentação
- No final é um *tradeoff* sobre o que escolher

Tabela de Páginas - Estrutura da Entrada

- **Page frame number** - identifica a página real
- **Bit de residência (presente/ausente)** - indica se a página é válida (está na RAM) quando o valor é 1
- **Bits de proteção** - 1 (leitura/escrita) ou 3 bits (leitura/escrita/execução)
- **Bit de modificação** - reflete o estado do conteúdo (bit dirty)
- **Bit de referenciamento** - se foi acessada recentemente
- **Bit de cache** - serve para desabilitar caching desta página

Tabela de Páginas - Estrutura da Entrada



Armazenamento da Tabela de Páginas

- Em qualquer sistema de paginação, duas questões precisam ser abordadas:
 - O mapeamento precisa ser rápido
 - Se o espaço de endereços virtuais for grande, a tabela de páginas também o será
- É muito fácil a busca tabela virar um gargalo
- Com 32 bits, e página de 4K, a tabela de páginas tem 1 milhão de páginas

Armazenamento da Tabela de Páginas

- Assim, deve-se decidir onde manter a tabela de páginas para uso:
 - **Array de Registradores** - casos de memória pequena
 - Tudo mantido no hardware
 - Na própria **memória principal** (mais utilizado)
 - A MMU gerencia utilizando registradores
 - Em uma **memória auxiliar** (cache) na MMU (ou memória associativa)
 - Utilizada para melhorar o desempenho da tabela na RAM

Armazenamento da Tabela de Páginas

- **Tabela na Memória Principal (1/2)**
 - Utiliza dois registradores
 - Registrador de base da tabela de página (PTBR)
 - Aponta para o início da tabela, indicando o endereço físico de memória onde a tabela está alocada
 - Registrador de tamanho da tabela de página (PTLR)
 - Indica o tamanho da tabela de página (número de entradas)

Armazenamento da Tabela de Páginas

- **Tabela na Memória Principal (2/2)**
 - Problema de se utilizar a tabela armazenada na própria memória principal:
 - Precisa de dois acessos na memória
 - Acesso para a tabela
 - Acesso para o dado
 - **Solução:** incluir hardware para caching de páginas (TLB)

Translation Lookaside Buffer (TLB)

- Módulo de hardware para mapear endereços físicos sem passar pela tabela de páginas
 - Costuma estar dentro MMU
- Forma de acelerar a paginação
 - Parte da observação que a maioria dos acessos ocorre em um número pequeno de páginas
 - Mesmas páginas sendo lidas/acessadas mais vezes

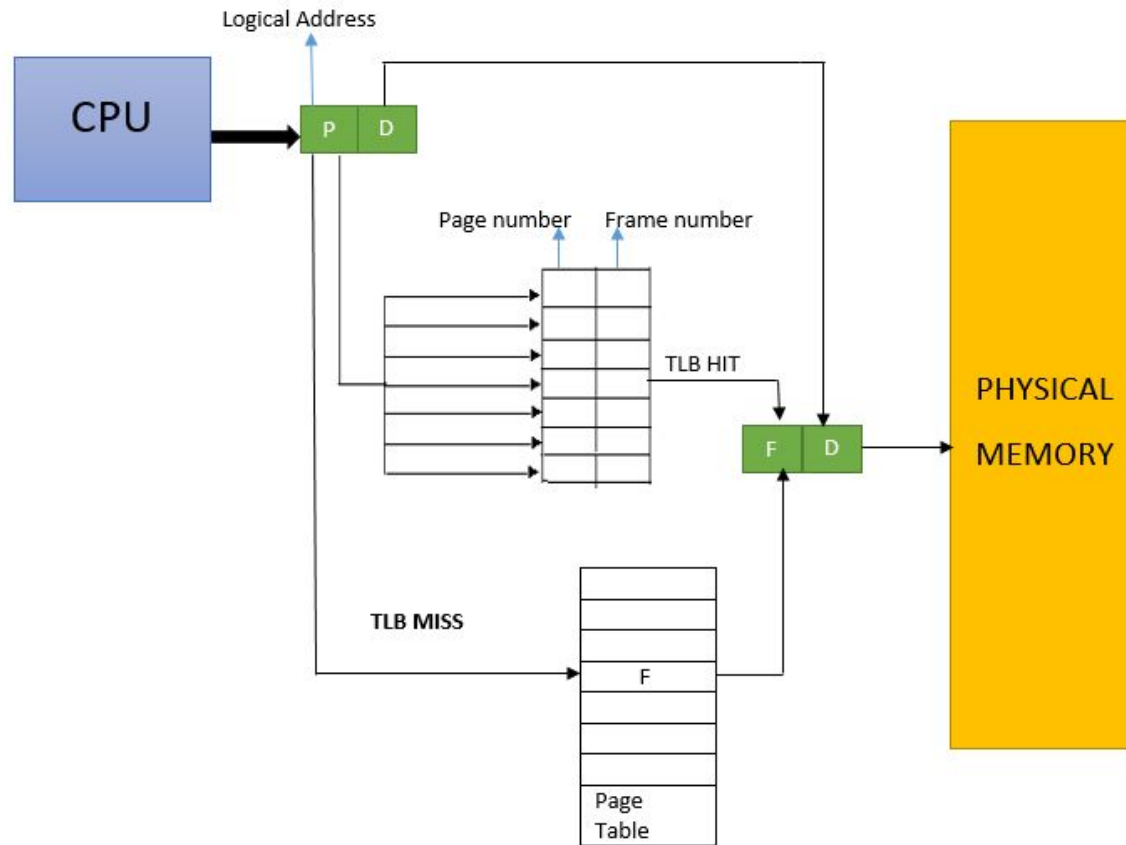
Translation Lookaside Buffer (TLB)

Válida	Página virtual	Modificada	Proteção	Quadro de página
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

Translation Lookaside Buffer (TLB)

- Pode ser implementada via hardware ou software
 - Em hardware é mais rápido, mas ocupa espaço de outras funções como cache
 - Em software, a TLB é carregada pelo SO
 - Quando ocorre uma falta (page fault), ao invés de recorrer à MMU, o problema volta para o Sistema Operacional
 - TLB precisa ser grande para reduzir a taxa de ausências

Translation Lookaside Buffer (TLB)



Translation Lookaside Buffer (TLB)

- Quanto a página não está na TLB, a MMU detecta e “destitui” uma entrada da TLB com a página recém buscada
 - Em uma busca subsequente, esta estará presente na TLB
- Quando uma entrada sai da TLB, o bit modificado é copiado na tabela de páginas da memória

Tipos de Page Fault (Falha de Página)

- **Soft Miss**

- Quando a página referenciada não está na TLB, mas está na RAM
- Basta atualizar a TLB

- **Hard Miss**

- A página não está na TLB e nem na memória física
- Precisa-se acessar e trazer do armazenamento secundário
 - Extremamente lento

Conclusão